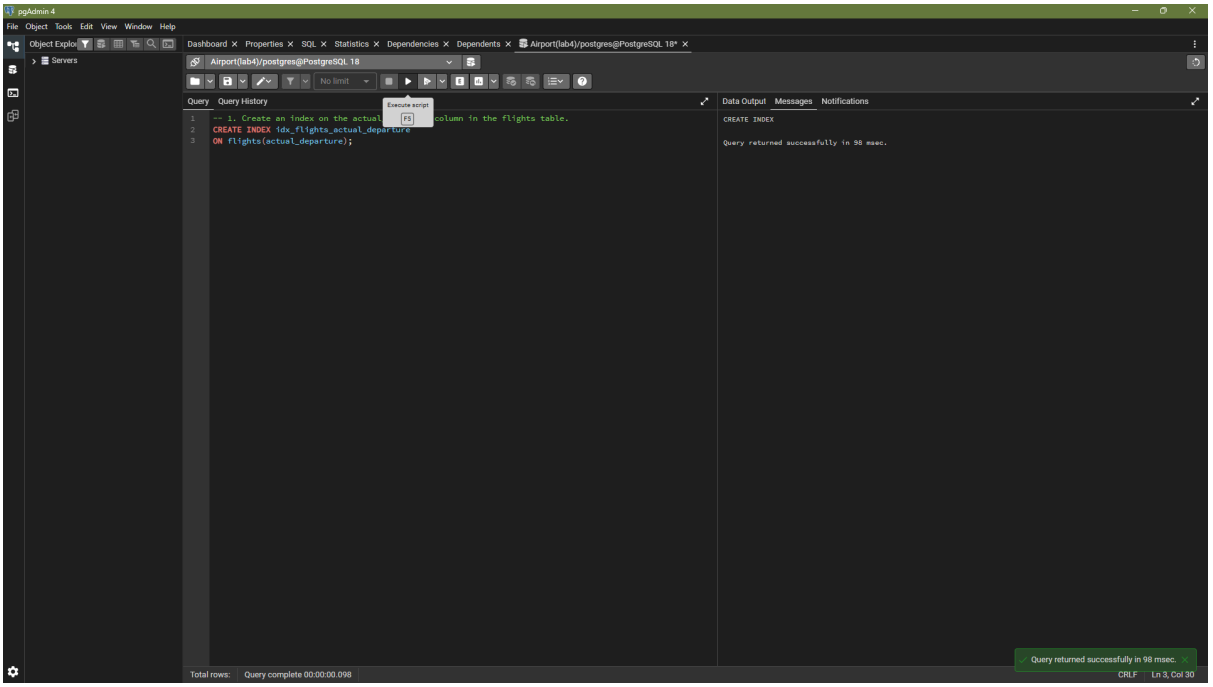
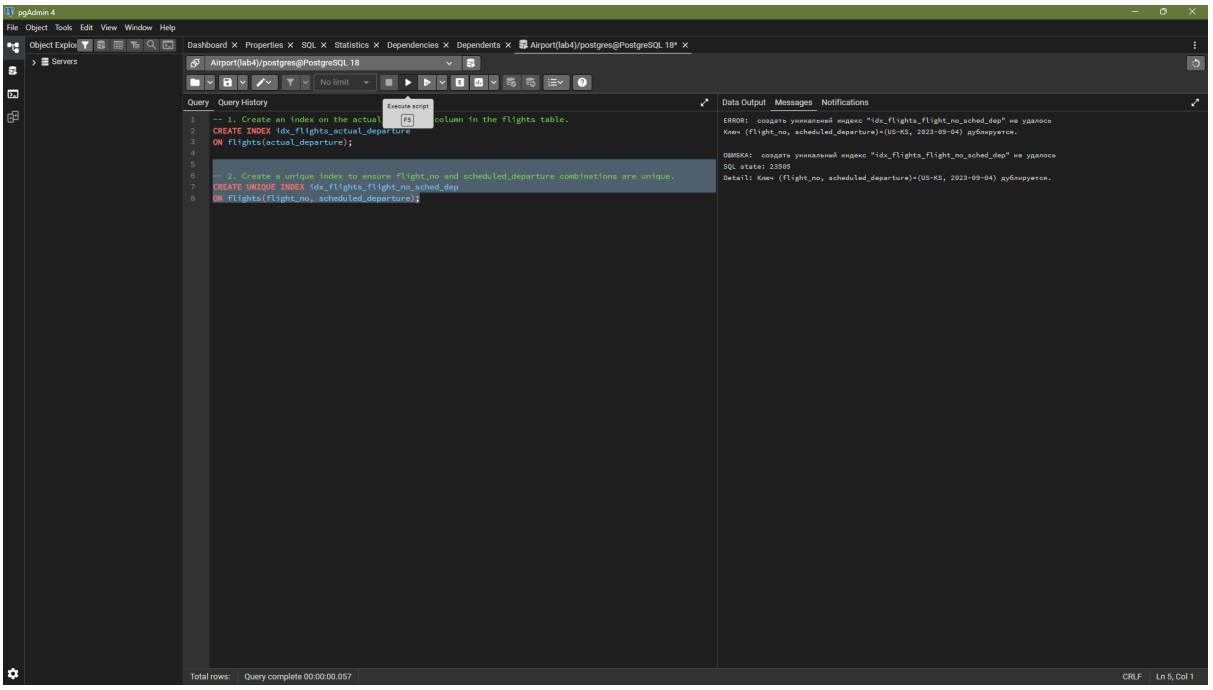
Lab 7

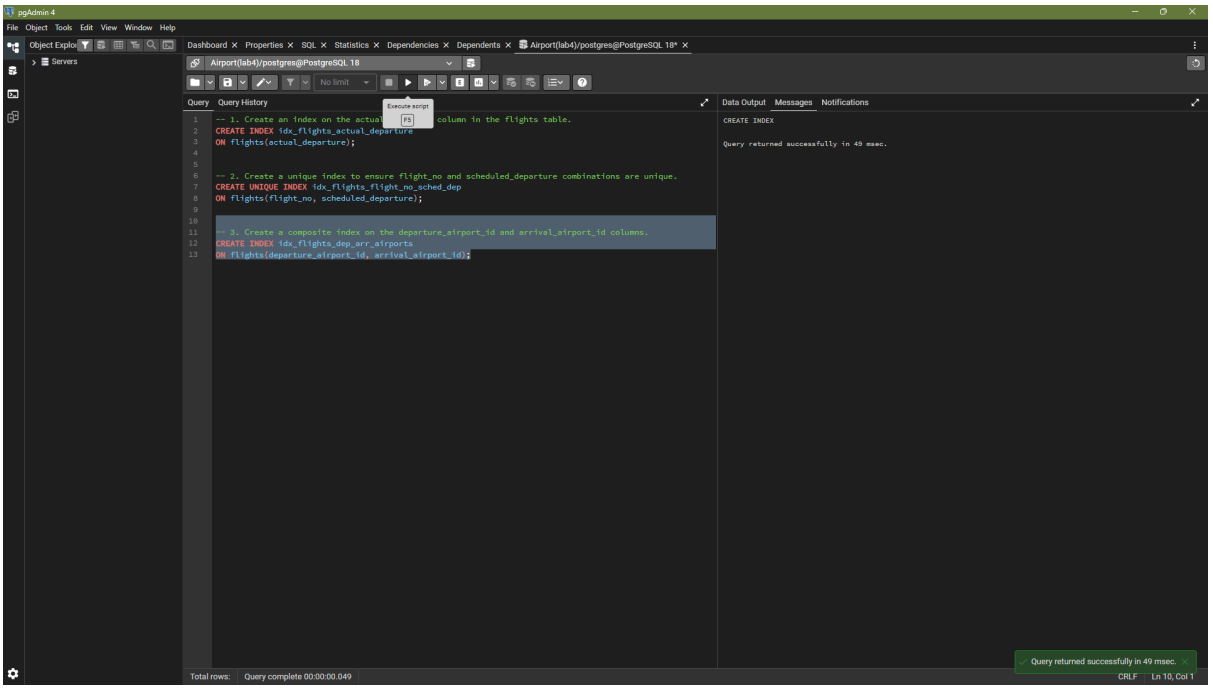
1.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Servers
Airport(lab4)/postgres@PostgreSQL 18
No limit
Execute script
F5
Query Query History
-- 1. Create an index on the actual column in the flights table.
CREATE INDEX idx_flights_actual_departure
ON flights(actual_departure);
Data Output Messages Notifications
CREATE INDEX
Query returned successfully in 98 msec.
Query returned successfully in 98 msec.
Total rows:
Query complete 00:00:00.098
CRLF
Ln 3, Col 30

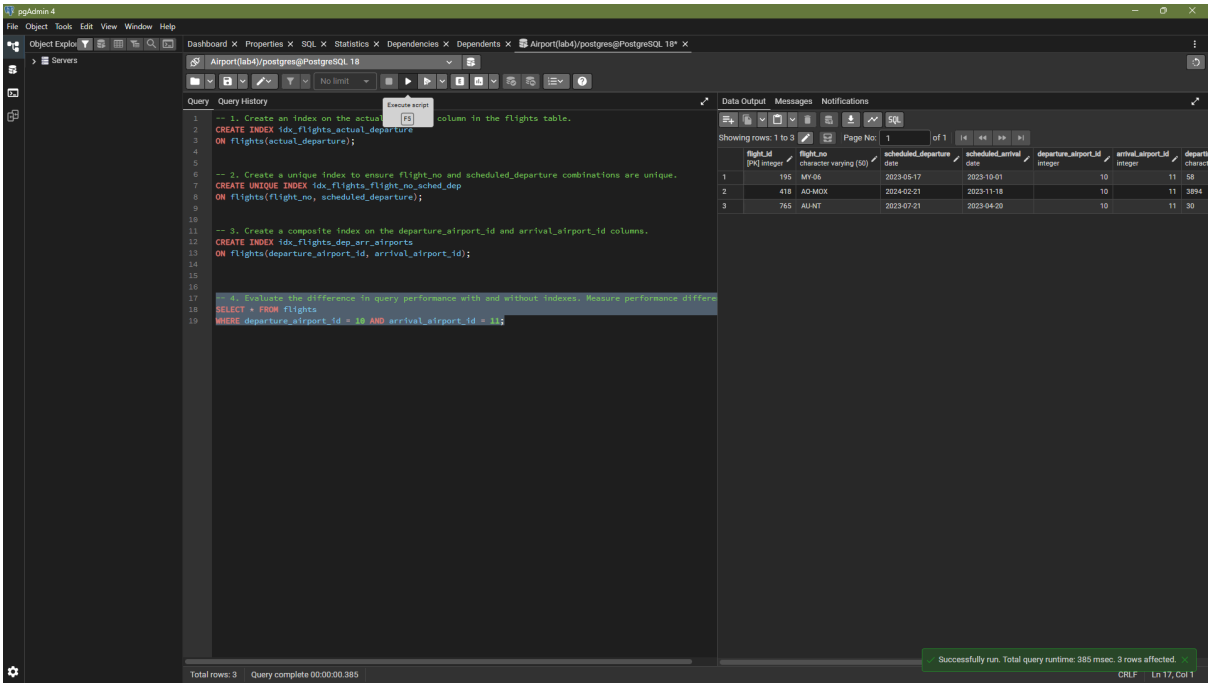
2.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
-- 1. Create an index on the actual column in the flights table.
CREATE INDEX idx_flights_actual_departure
ON flights(actual_departure);
-- 2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.
CREATE UNIQUE INDEX idx_flights_flight_no_sched_dep
ON flights(flight_no, scheduled_departure);
Data Output Messages Notifications
ERROR: создать уникальный индекс "idx_flights_flight_no_sched_dep" не удалось
Ключ (flight_no, scheduled_departure)=(US-K5, 2023-09-04) дублируется.
ОШИБКА: создать уникальный индекс "idx_flights_flight_no_sched_dep" не удалось
SQL state: 23505
Detail: Ключ (flight_no, scheduled_departure)=(US-K5, 2023-09-04) дублируется.
Total rows: Query complete 00:00:00.057
CRLF Ln 5, Col 1

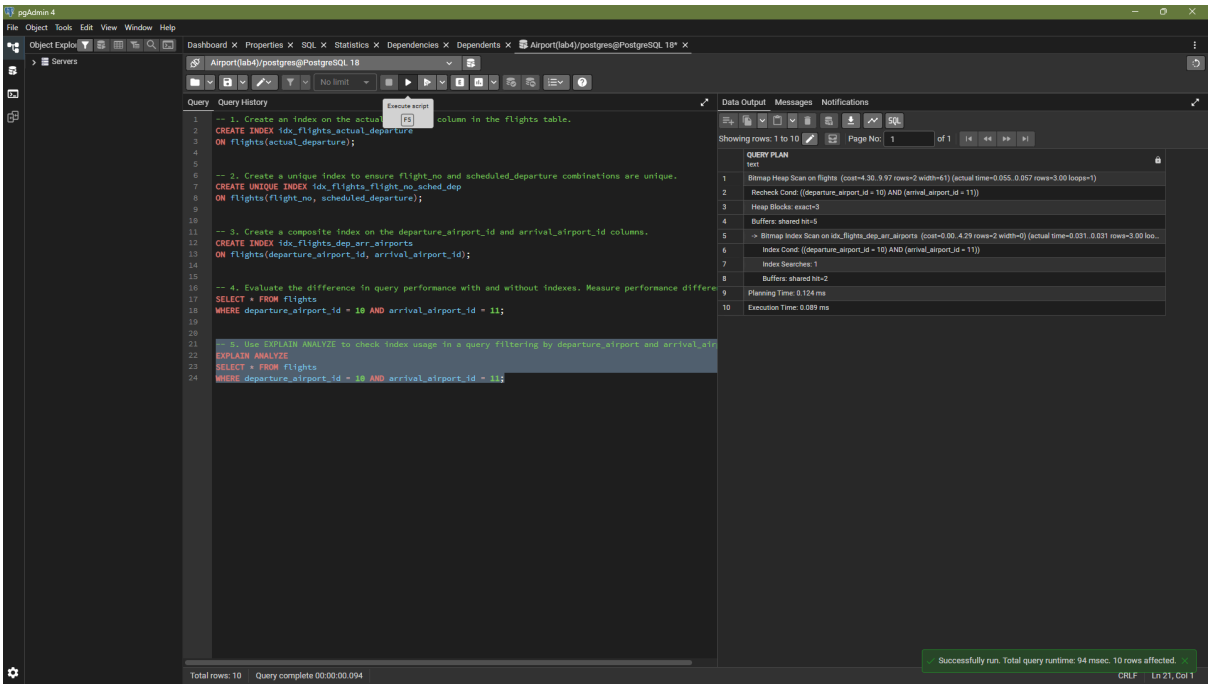
3.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
-- 1. Create an index on the actual column in the flights table.
CREATE INDEX idx_flights_actual_departure
ON flights(actual_departure);
-- 2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.
CREATE UNIQUE INDEX idx_flights_flight_no_sched_dep
ON flights(flight_no, scheduled_departure);
-- 3. Create a composite index on the departure_airport_id and arrival_airport_id columns.
CREATE INDEX idx_flights_dep_arr_airports
ON flights(departure_airport_id, arrival_airport_id);
Data Output Messages Notifications
CREATE INDEX
Query returned successfully in 49 msec.
Query returned successfully in 49 msec.
Total rows: Query complete 00:00:00.049
CRLF Ln 10, Col 1

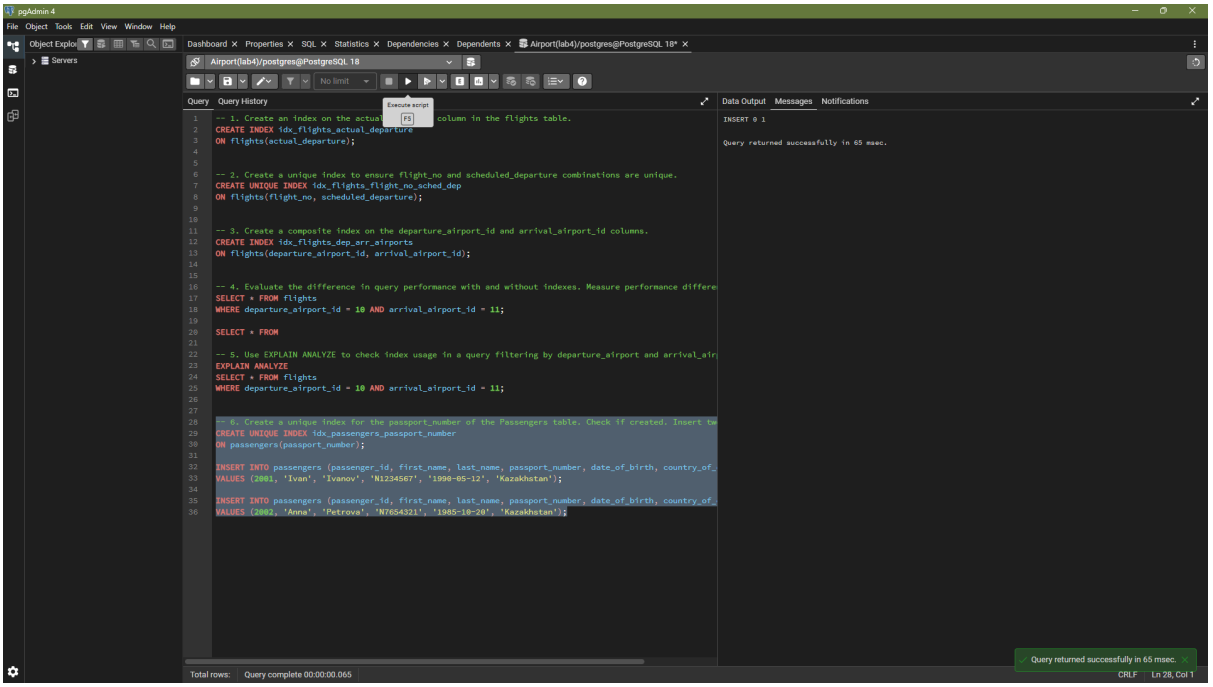
4.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
-- 1. Create an index on the actual column in the flights table.
CREATE INDEX idx_flights_actual_departure
ON flights(actual_departure);
-- 2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.
CREATE UNIQUE INDEX idx_flights_flight_no_sched_dep
ON flights(flight_no, scheduled_departure);
-- 3. Create a composite index on the departure_airport_id and arrival_airport_id columns.
CREATE INDEX idx_flights_dep_arr_airports
ON flights(departure_airport_id, arrival_airport_id);
-- 4. Evaluate the difference in query performance with and without indexes. Measure performance differe
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
Data Output Messages Notifications
Showing rows: 1 to 3 Page No: 1 of 1
flight_id [PK] integer
flight_no character varying (50)
scheduled_departure date
scheduled_arrival date
departure_airport_id integer
arrival_airport_id integer
1 195 MY-06 2023-05-17 2023-10-01 10 11 58
2 418 AO-MOX 2024-02-21 2023-11-18 10 11 3894
3 765 AU-NT 2023-07-21 2023-04-20 10 11 30
Successfully run. Total query runtime: 385 msec. 3 rows affected.
Total rows: 3 Query complete 00:00:00.385
CRLF Ln 17, Col 1

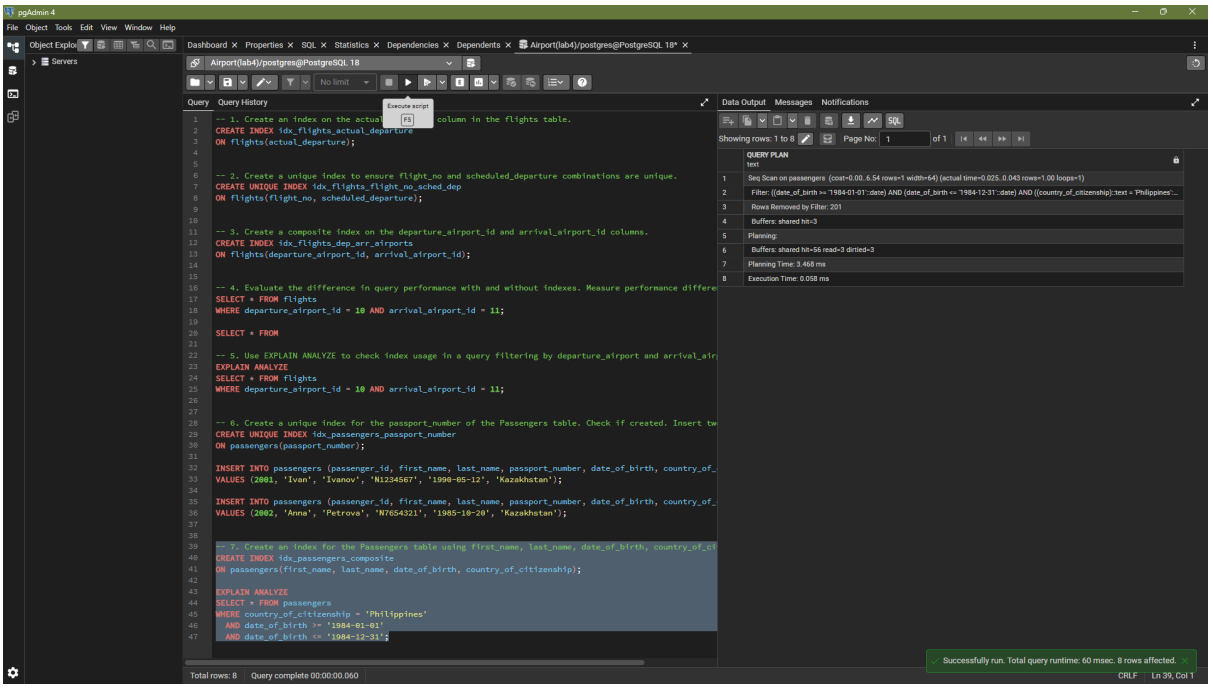
5.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
-- 1. Create an index on the actual column in the flights table.
CREATE INDEX idx_flights_actual_departure
ON flights(actual_departure);
-- 2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.
CREATE UNIQUE INDEX idx_flights_flight_no_sched_dep
ON flights(flight_no, scheduled_departure);
-- 3. Create a composite index on the departure_airport_id and arrival_airport_id columns.
CREATE INDEX idx_flights_dep_arr_airports
ON flights(departure_airport_id, arrival_airport_id);
-- 4. Evaluate the difference in query performance with and without indexes. Measure performance differe
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
-- 5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_air
EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
Data Output Messages Notifications
Showing rows: 1 to 10 Page No: 1 of 1
QUERY PLAN text
1 Bitmap Heap Scan on flights (cost=4.30..9.97 rows=2 width=61) (actual time=0.055..0.057 rows=3.00 loops=1)
2 Recheck Cond: ((departure_airport_id = 10) AND (arrival_airport_id = 11))
3 Heap Blocks: exact=3
4 Buffers: shared hit=5
5 -> Bitmap Index Scan on idx_flights_dep_arr_airports (cost=0.00..4.29 rows=2 width=0) (actual time=0.031..0.031 rows=3.00 loo...
6 Index Cond: ((departure_airport_id = 10) AND (arrival_airport_id = 11))
7 Index Searches: 1
8 Buffers: shared hit=2
9 Planning Time: 0.124 ms
10 Execution Time: 0.089 ms
Successfully run. Total query runtime: 94 msec. 10 rows affected.
Total rows: 10 Query complete 00:00:00.094
CRLF Ln 21, Col 1

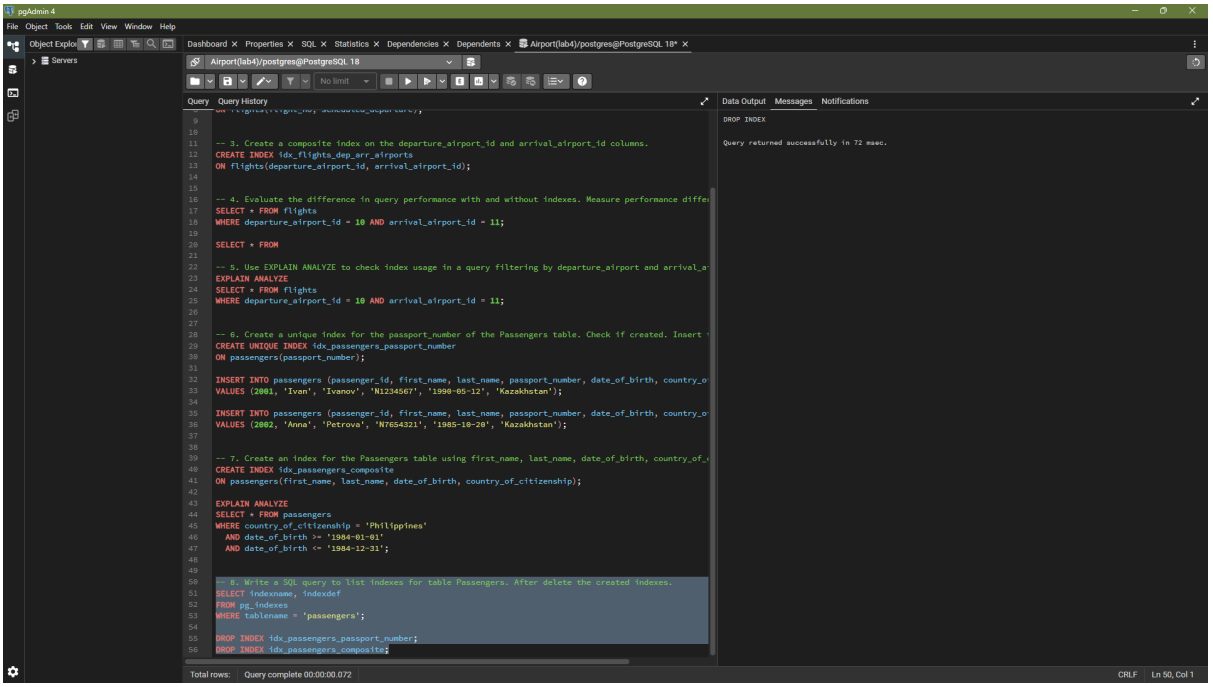
6.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Execute script
-- 1. Create an index on the actual column in the flights table.
CREATE INDEX idx_flights_actual_departure
ON flights(actual_departure);
-- 2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.
CREATE UNIQUE INDEX idx_flights_flight_no_sched_dep
ON flights(flight_no, scheduled_departure);
-- 3. Create a composite index on the departure_airport_id and arrival_airport_id columns.
CREATE INDEX idx_flights_dep_arr_airports
ON flights(departure_airport_id, arrival_airport_id);
-- 4. Evaluate the difference in query performance with and without indexes. Measure performance differe
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
SELECT * FROM
-- 5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_air
EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
-- 6. Create a unique index for the passport_number of the Passengers table. Check if created. Insert tw
CREATE UNIQUE INDEX idx_passengers_passport_number
ON passengers(passport_number);
INSERT INTO passengers (passenger_id, first_name, last_name, passport_number, date_of_birth, country_of_
VALUES (2001, 'Ivan', 'Ivanov', 'N1234567', '1990-05-12', 'Kazakhstan');
INSERT INTO passengers (passenger_id, first_name, last_name, passport_number, date_of_birth, country_of_
VALUES (2002, 'Anna', 'Petrova', 'N7654321', '1985-10-20', 'Kazakhstan');
Data Output Messages Notifications
INSERT 0 1
Query returned successfully in 65 msec.
Query returned successfully in 65 msec.
Total rows: Query complete 00:00:00.065
CRLF Ln 28, Col 1

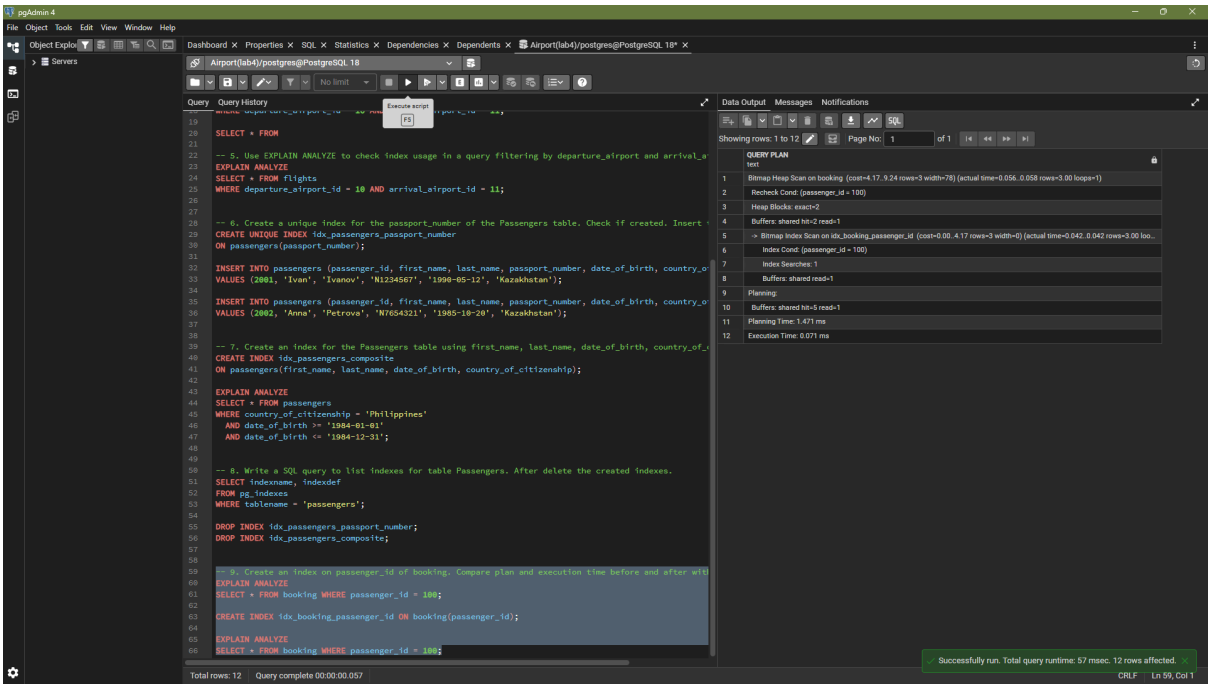
7.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Execute script
-- 1. Create an index on the actual column in the flights table.
CREATE INDEX idx_flights_actual_departure
ON flights(actual_departure);
-- 2. Create a unique index to ensure flight_no and scheduled_departure combinations are unique.
CREATE UNIQUE INDEX idx_flights_flight_no_sched_dep
ON flights(flight_no, scheduled_departure);
-- 3. Create a composite index on the departure_airport_id and arrival_airport_id columns.
CREATE INDEX idx_flights_dep_arr_airports
ON flights(departure_airport_id, arrival_airport_id);
-- 4. Evaluate the difference in query performance with and without indexes. Measure performance differe
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
SELECT * FROM
-- 5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_air
EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
-- 6. Create a unique index for the passport_number of the Passengers table. Check if created. Insert tw
CREATE UNIQUE INDEX idx_passengers_passport_number
ON passengers(passport_number);
INSERT INTO passengers (passenger_id, first_name, last_name, passport_number, date_of_birth, country_of_
VALUES (2001, 'Ivan', 'Ivanov', 'N1234567', '1990-05-12', 'Kazakhstan');
INSERT INTO passengers (passenger_id, first_name, last_name, passport_number, date_of_birth, country_of_
VALUES (2002, 'Anna', 'Petrova', 'N7654321', '1985-10-20', 'Kazakhstan');
-- 7. Create an index for the Passengers table using first_name, last_name, date_of_birth, country_of_ci
CREATE INDEX idx_passengers_composite
ON passengers(first_name, last_name, date_of_birth, country_of_citizenship);
EXPLAIN ANALYZE
SELECT * FROM passengers
WHERE country_of_citizenship = 'Philippines'
AND date_of_birth >= '1984-01-01'
AND date_of_birth <= '1984-12-31';
Data Output Messages Notifications
Showing rows: 1 to 8 Page No: 1 of 1
QUERY PLAN
text
1 Seq Scan on passengers (cost=0.00..6.54 rows=1 width=64) (actual time=0.025..0.043 rows=1.00 loops=1)
2 Filter: ((date_of_birth >= '1984-01-01'::date) AND (date_of_birth <= '1984-12-31'::date) AND ((country_of_citizenship)::text = 'Philippines'...
3 Rows Removed by Filter: 201
4 Buffers: shared hit=3
5 Planning:
6 Buffers: shared hit=56 read=3 dirtied=3
7 Planning Time: 3.468 ms
8 Execution Time: 0.058 ms
Successfully run. Total query runtime: 60 msec. 8 rows affected.
Total rows: 8 Query complete 00:00:00.060
CRLF Ln 39, Col 1

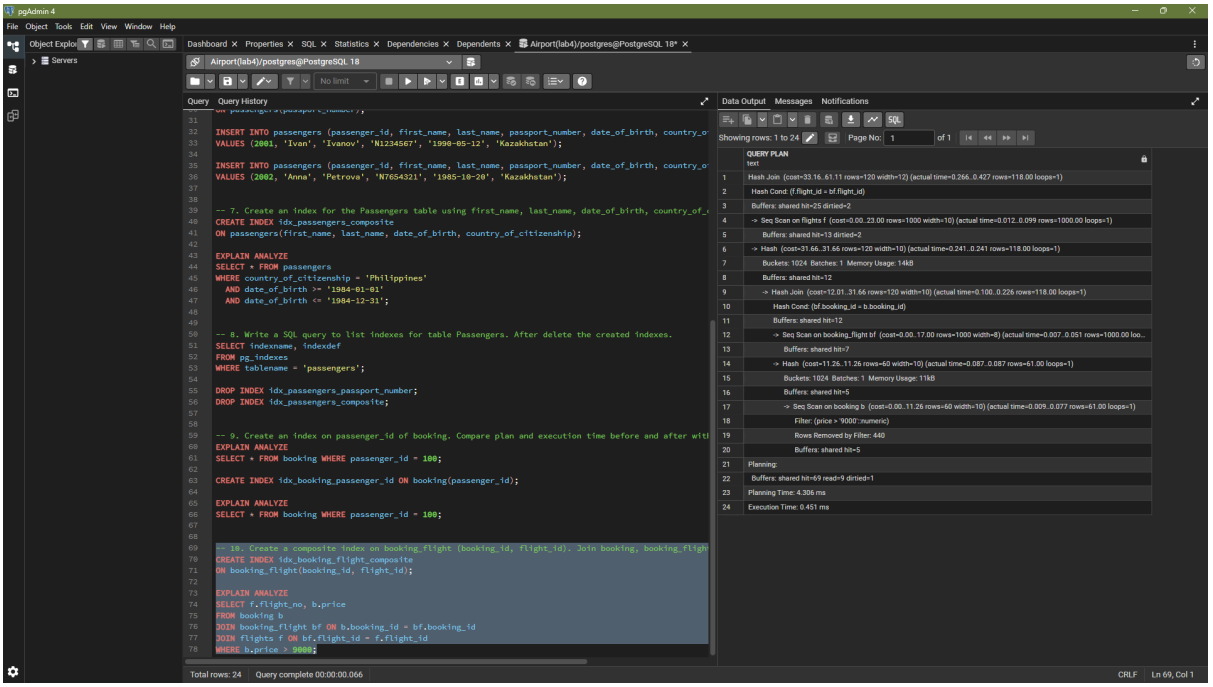
8.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Data Output Messages Notifications
DROP INDEX
Query returned successfully in 72 msec.
-- 3. Create a composite index on the departure_airport_id and arrival_airport_id columns.
CREATE INDEX idx_flights_dep_arr_airports
ON flights(departure_airport_id, arrival_airport_id);
-- 4. Evaluate the difference in query performance with and without indexes. Measure performance diffe
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
SELECT * FROM
-- 5. Use EXPLAIN ANALYZE to check index usage in a query filtering by departure_airport and arrival_a
EXPLAIN ANALYZE
SELECT * FROM flights
WHERE departure_airport_id = 10 AND arrival_airport_id = 11;
-- 6. Create a unique index for the passport_number of the Passengers table. Check if created. Insert
CREATE UNIQUE INDEX idx_passengers_passport_number
ON passengers(passport_number);
INSERT INTO passengers (passenger_id, first_name, last_name, passport_number, date_of_birth, country_o
VALUES (2001, 'Ivan', 'Ivanov', 'N1234567', '1990-05-12', 'Kazakhstan');
INSERT INTO passengers (passenger_id, first_name, last_name, passport_number, date_of_birth, country_o
VALUES (2002, 'Anna', 'Petrova', 'N7654321', '1985-10-20', 'Kazakhstan');
-- 7. Create an index for the Passengers table using first_name, last_name, date_of_birth, country_of_
CREATE INDEX idx_passengers_composite
ON passengers(first_name, last_name, date_of_birth, country_of_citizenship);
EXPLAIN ANALYZE
SELECT * FROM passengers
WHERE country_of_citizenship = 'Philippines'
AND date_of_birth >= '1984-01-01'
AND date_of_birth <= '1984-12-31';
-- 8. Write a SQL query to list indexes for table Passengers. After delete the created indexes.
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';
DROP INDEX idx_passengers_passport_number;
DROP INDEX idx_passengers_composite;
Total rows: Query complete 00:00:00.072
CRLF Ln 50, Col 1

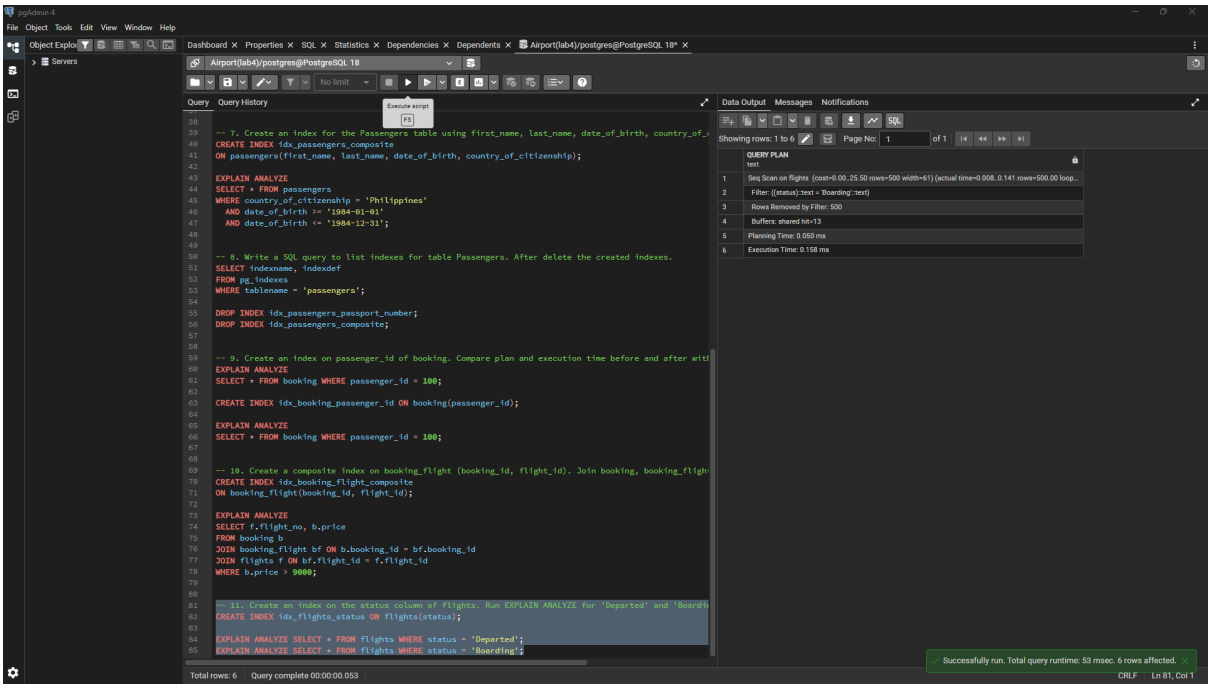
9.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Execute script F5
Data Output Messages Notifications
Showing rows: 1 to 12 Page No: 1 of 1
QUERY PLAN
text
1 Bitmap Heap Scan on booking (cost=4.17..9.24 rows=3 width=78) (actual time=0.056..0.058 rows=3.00 loops=1)
2 Recheck Cond: (passenger_id = 100)
3 Heap Blocks: exact=2
4 Buffers: shared hit=2 read=1
5 -> Bitmap Index Scan on idx_booking_passenger_id (cost=0.00..4.17 rows=3 width=0) (actual time=0.042..0.042 rows=3.00 loo...
6 Index Cond: (passenger_id = 100)
7 Index Searches: 1
8 Buffers: shared read=1
9 Planning:
10 Buffers: shared hit=5 read=1
11 Planning Time: 1.471 ms
12 Execution Time: 0.071 ms
-- 6. Create a unique index for the passport_number of the Passengers table. Check if created. Insert
CREATE UNIQUE INDEX idx_passengers_passport_number
ON passengers(passport_number);
INSERT INTO passengers (passenger_id, first_name, last_name, passport_number, date_of_birth, country_o
VALUES (2001, 'Ivan', 'Ivanov', 'N1234567', '1990-05-12', 'Kazakhstan');
INSERT INTO passengers (passenger_id, first_name, last_name, passport_number, date_of_birth, country_o
VALUES (2002, 'Anna', 'Petrova', 'N7654321', '1985-10-20', 'Kazakhstan');
-- 7. Create an index for the Passengers table using first_name, last_name, date_of_birth, country_of_
CREATE INDEX idx_passengers_composite
ON passengers(first_name, last_name, date_of_birth, country_of_citizenship);
EXPLAIN ANALYZE
SELECT * FROM passengers
WHERE country_of_citizenship = 'Philippines'
AND date_of_birth >= '1984-01-01'
AND date_of_birth <= '1984-12-31';
-- 8. Write a SQL query to list indexes for table Passengers. After delete the created indexes.
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';
DROP INDEX idx_passengers_passport_number;
DROP INDEX idx_passengers_composite;
-- 9. Create an index on passenger_id of booking. Compare plan and execution time before and after wit
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
CREATE INDEX idx_booking_passenger_id ON booking(passenger_id);
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
Successfully run. Total query runtime: 57 msec. 12 rows affected.
Total rows: 12 Query complete 00:00:00.057
CRLF Ln 59, Col 1

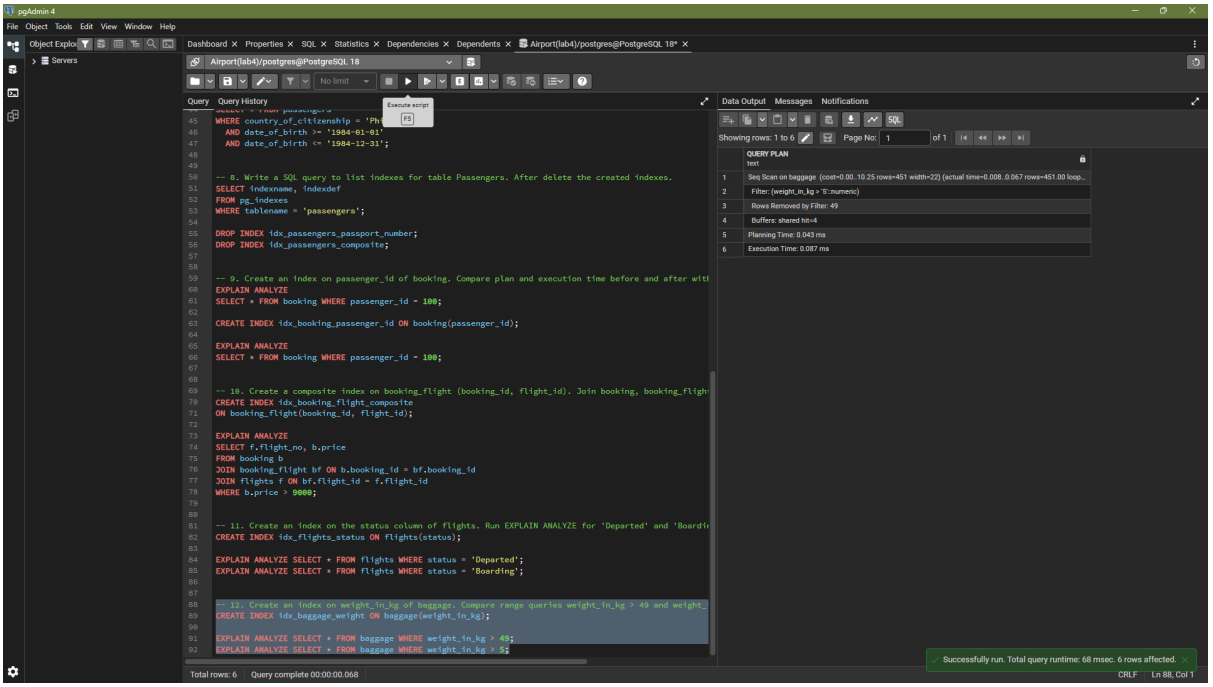
10.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Data Output Messages Notifications
QUERY PLAN
text
Hash Join (cost=33.16..61.11 rows=120 width=12) (actual time=0.266..0.427 rows=118.00 loops=1)
Hash Cond: (f.flight_id = bf.flight_id)
Buffers: shared hit=25 dirtied=2
-> Seq Scan on flights f (cost=0.00..23.00 rows=1000 width=10) (actual time=0.012..0.099 rows=1000.00 loops=1)
Buffers: shared hit=13 dirtied=2
-> Hash (cost=31.66..31.66 rows=120 width=10) (actual time=0.241..0.241 rows=118.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 14kB
Buffers: shared hit=12
-> Hash Join (cost=12.01..31.66 rows=120 width=10) (actual time=0.100..0.226 rows=118.00 loops=1)
Hash Cond: (bf.booking_id = b.booking_id)
Buffers: shared hit=12
-> Seq Scan on booking_flight bf (cost=0.00..17.00 rows=1000 width=8) (actual time=0.007..0.051 rows=1000.00 loo...
Buffers: shared hit=7
-> Hash (cost=11.26..11.26 rows=60 width=10) (actual time=0.087..0.087 rows=61.00 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 11kB
Buffers: shared hit=5
-> Seq Scan on booking b (cost=0.00..11.26 rows=60 width=10) (actual time=0.009..0.077 rows=61.00 loops=1)
Filter: (price > '9000'::numeric)
Rows Removed by Filter: 440
Buffers: shared hit=5
Planning:
Buffers: shared hit=69 read=9 dirtied=1
Planning Time: 4.306 ms
Execution Time: 0.451 ms
Total rows: 24 Query complete 00:00:00.066
CRLF Ln 69, Col 1

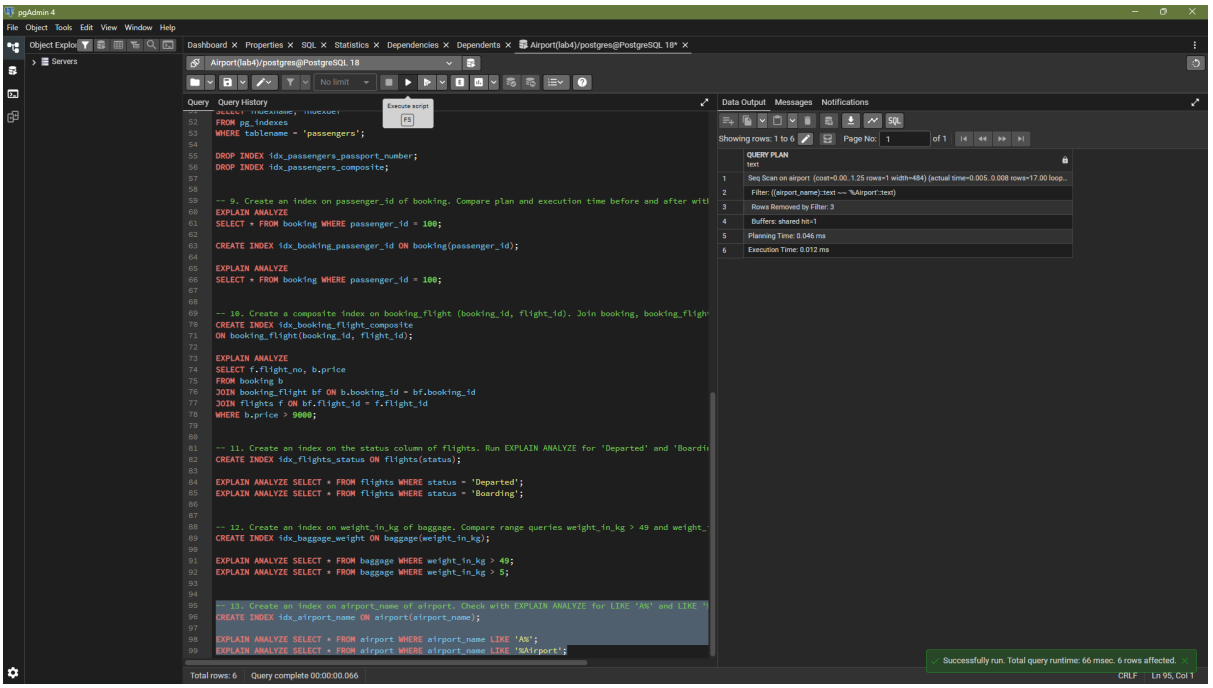
11.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Execute script
F5
Query Query History
Data Output Messages Notifications
QUERY PLAN
text
Seq Scan on flights (cost=0.00..25.50 rows=500 width=61) (actual time=0.008..0.141 rows=500.00 loop...
Filter: ((status)::text = 'Boarding'::text)
Rows Removed by Filter: 500
Buffers: shared hit=13
Planning Time: 0.050 ms
Execution Time: 0.158 ms
Successfully run. Total query runtime: 53 msec. 6 rows affected.
Total rows: 6 Query complete 00:00:00.053
CRLF Ln 81, Col 1

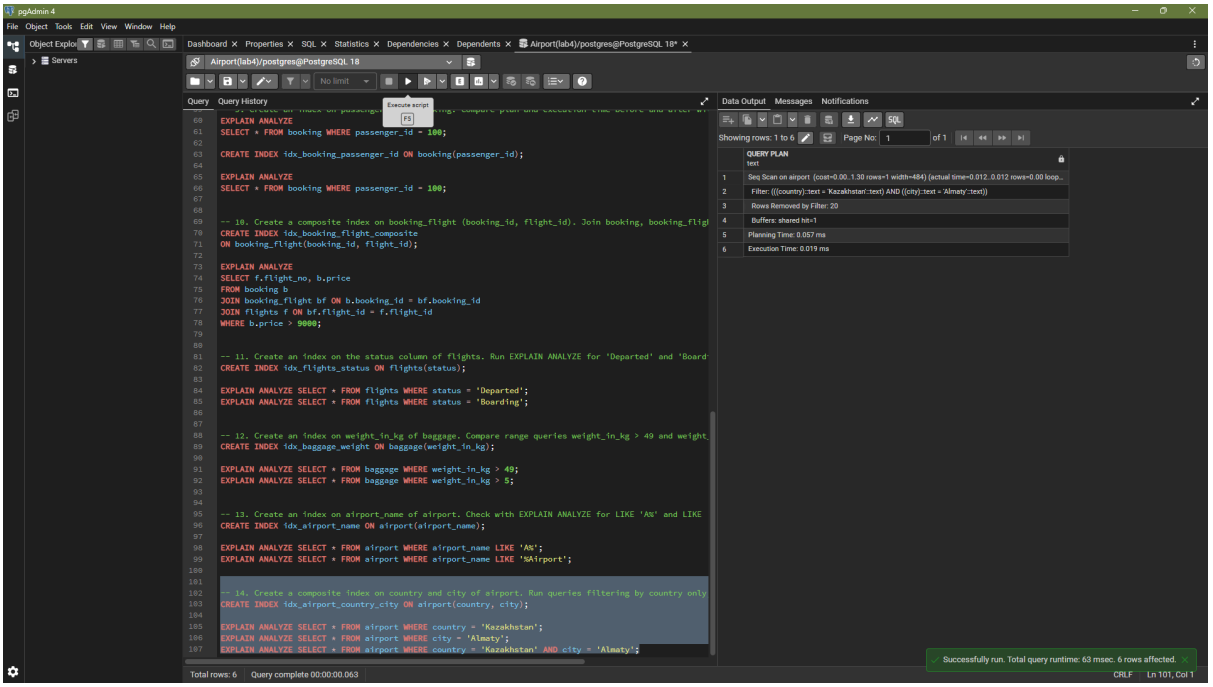
12.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Data Output Messages Notifications
WHERE country_of_citizenship = 'Phi
AND date_of_birth >= '1984-01-01'
AND date_of_birth <= '1984-12-31';
-- 8. Write a SQL query to list indexes for table Passengers. After delete the created indexes.
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';
DROP INDEX idx_passengers_passport_number;
DROP INDEX idx_passengers_composite;
-- 9. Create an index on passenger_id of booking. Compare plan and execution time before and after with
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
CREATE INDEX idx_booking_passenger_id ON booking(passenger_id);
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
-- 10. Create a composite index on booking_flight (booking_id, flight_id). Join booking, booking_flight
CREATE INDEX idx_booking_flight_composite
ON booking_flight(booking_id, flight_id);
EXPLAIN ANALYZE
SELECT f.flight_no, b.price
FROM booking b
JOIN booking_flight bf ON b.booking_id = bf.booking_id
JOIN flights f ON bf.flight_id = f.flight_id
WHERE b.price > 9000;
-- 11. Create an index on the status column of flights. Run EXPLAIN ANALYZE for 'Departed' and 'Boardi
CREATE INDEX idx_flights_status ON flights(status);
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Departed';
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Boarding';
-- 12. Create an index on weight_in_kg of baggage. Compare range queries weight_in_kg > 49 and weight_
CREATE INDEX idx_baggage_weight ON baggage(weight_in_kg);
EXPLAIN ANALYZE SELECT * FROM baggage WHERE weight_in_kg > 49;
EXPLAIN ANALYZE SELECT * FROM baggage WHERE weight_in_kg > 5;
QUERY PLAN
text
1 Seq Scan on baggage (cost=0.00..10.25 rows=451 width=22) (actual time=0.008..0.067 rows=451.00 loop...
2 Filter: (weight_in_kg > '5'::numeric)
3 Rows Removed by Filter: 49
4 Buffers: shared hit=4
5 Planning Time: 0.043 ms
6 Execution Time: 0.087 ms
Successfully run. Total query runtime: 68 msec. 6 rows affected.
Total rows: 6 Query complete 00:00:00.068
CRLF Ln 88, Col 1

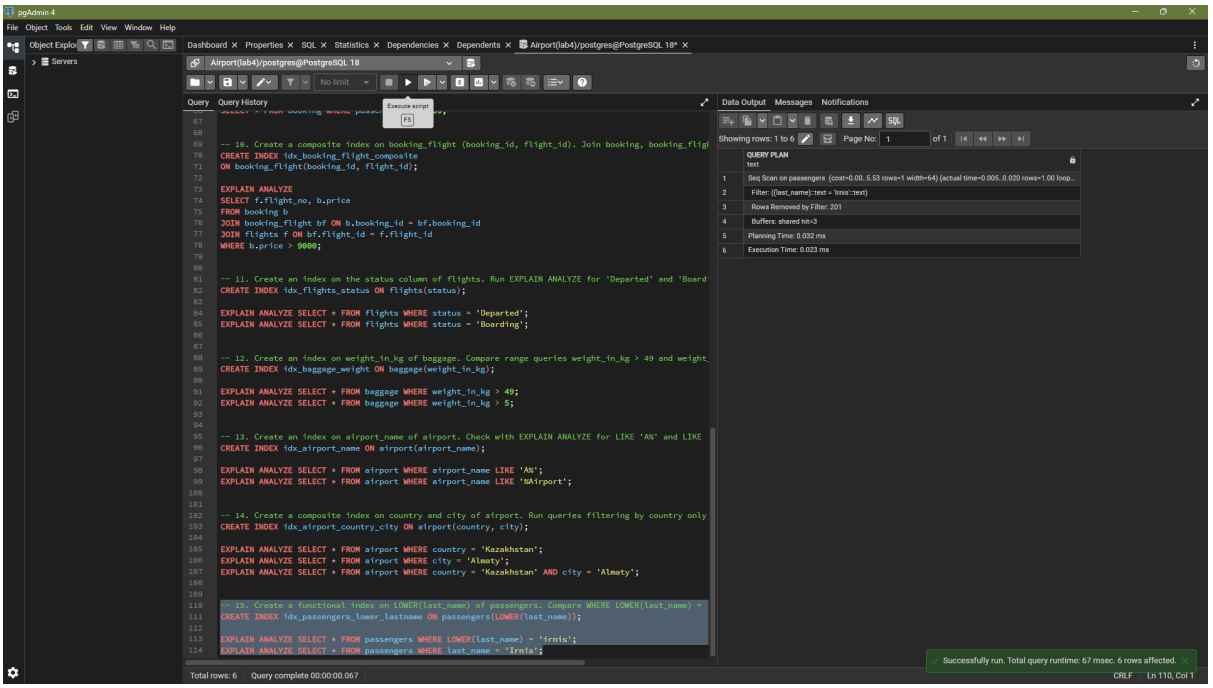
13.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Data Output Messages Notifications
FROM pg_indexes
WHERE tablename = 'passengers';
DROP INDEX idx_passengers_passport_number;
DROP INDEX idx_passengers_composite;
-- 9. Create an index on passenger_id of booking. Compare plan and execution time before and after with
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
CREATE INDEX idx_booking_passenger_id ON booking(passenger_id);
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
-- 10. Create a composite index on booking_flight (booking_id, flight_id). Join booking, booking_flight
CREATE INDEX idx_booking_flight_composite
ON booking_flight(booking_id, flight_id);
EXPLAIN ANALYZE
SELECT f.flight_no, b.price
FROM booking b
JOIN booking_flight bf ON b.booking_id = bf.booking_id
JOIN flights f ON bf.flight_id = f.flight_id
WHERE b.price > 9000;
-- 11. Create an index on the status column of flights. Run EXPLAIN ANALYZE for 'Departed' and 'Boardi
CREATE INDEX idx_flights_status ON flights(status);
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Departed';
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Boarding';
-- 12. Create an index on weight_in_kg of baggage. Compare range queries weight_in_kg > 49 and weight_
CREATE INDEX idx_baggage_weight ON baggage(weight_in_kg);
EXPLAIN ANALYZE SELECT * FROM baggage WHERE weight_in_kg > 49;
EXPLAIN ANALYZE SELECT * FROM baggage WHERE weight_in_kg > 5;
-- 13. Create an index on airport_name of airport. Check with EXPLAIN ANALYZE for LIKE 'A%' and LIKE '%
CREATE INDEX idx_airport_name ON airport(airport_name);
EXPLAIN ANALYZE SELECT * FROM airport WHERE airport_name LIKE 'A%';
EXPLAIN ANALYZE SELECT * FROM airport WHERE airport_name LIKE '%Airport';
QUERY PLAN
text
1 Seq Scan on airport (cost=0.00..1.25 rows=1 width=484) (actual time=0.005..0.008 rows=17.00 loop...
2 Filter: ((airport_name)::text ~~ '%Airport'::text)
3 Rows Removed by Filter: 3
4 Buffers: shared hit=1
5 Planning Time: 0.046 ms
6 Execution Time: 0.012 ms
Successfully run. Total query runtime: 66 msec. 6 rows affected.
Total rows: 6 Query complete 00:00:00.066
CRLF Ln 95, Col 1

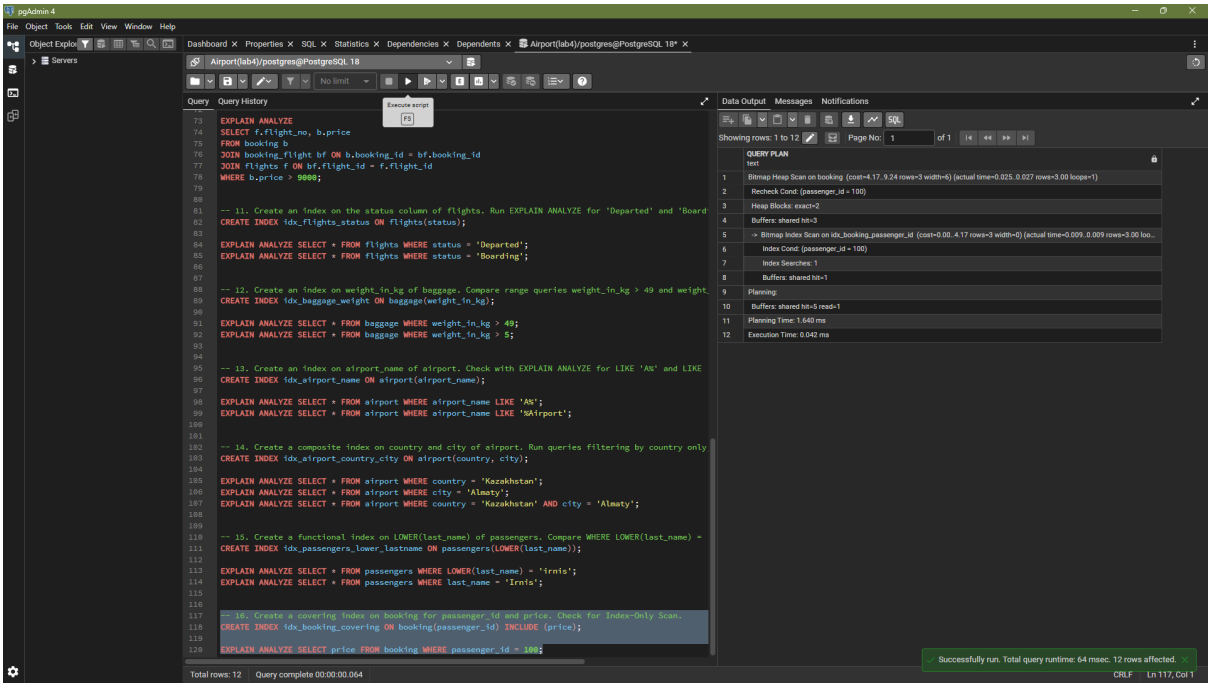
14.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Execute script
F5
Data Output Messages Notifications
Showing rows: 1 to 6 Page No: 1 of 1
QUERY PLAN
text
1 Seq Scan on airport (cost=0.00..1.30 rows=1 width=484) (actual time=0.012..0.012 rows=0.00 loop...
2 Filter: (((country)::text = Kazakhstan'::text) AND ((city)::text = 'Almaty'::text))
3 Rows Removed by Filter: 20
4 Buffers: shared hit=1
5 Planning Time: 0.057 ms
6 Execution Time: 0.019 ms
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
CREATE INDEX idx_booking_passenger_id ON booking(passenger_id);
EXPLAIN ANALYZE
SELECT * FROM booking WHERE passenger_id = 100;
-- 10. Create a composite index on booking_flight (booking_id, flight_id). Join booking, booking_flig
CREATE INDEX idx_booking_flight_composite
ON booking_flight(booking_id, flight_id);
EXPLAIN ANALYZE
SELECT f.flight_no, b.price
FROM booking b
JOIN booking_flight bf ON b.booking_id = bf.booking_id
JOIN flights f ON bf.flight_id = f.flight_id
WHERE b.price > 9000;
-- 11. Create an index on the status column of flights. Run EXPLAIN ANALYZE for 'Departed' and 'Board
CREATE INDEX idx_flights_status ON flights(status);
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Departed';
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Boarding';
-- 12. Create an index on weight_in_kg of baggage. Compare range queries weight_in_kg > 49 and weight_
CREATE INDEX idx_baggage_weight ON baggage(weight_in_kg);
EXPLAIN ANALYZE SELECT * FROM baggage WHERE weight_in_kg > 49;
EXPLAIN ANALYZE SELECT * FROM baggage WHERE weight_in_kg > 5;
-- 13. Create an index on airport_name of airport. Check with EXPLAIN ANALYZE for LIKE 'A%' and LIKE
CREATE INDEX idx_airport_name ON airport(airport_name);
EXPLAIN ANALYZE SELECT * FROM airport WHERE airport_name LIKE 'A%';
EXPLAIN ANALYZE SELECT * FROM airport WHERE airport_name LIKE '%Airport';
-- 14. Create a composite index on country and city of airport. Run queries filtering by country only
CREATE INDEX idx_airport_country_city ON airport(country, city);
EXPLAIN ANALYZE SELECT * FROM airport WHERE country = 'Kazakhstan';
EXPLAIN ANALYZE SELECT * FROM airport WHERE city = 'Almaty';
EXPLAIN ANALYZE SELECT * FROM airport WHERE country = 'Kazakhstan' AND city = 'Almaty';
Successfully run. Total query runtime: 63 msec. 6 rows affected.
Total rows: 6 Query complete 00:00:00.063
CRLF Ln 101, Col 1

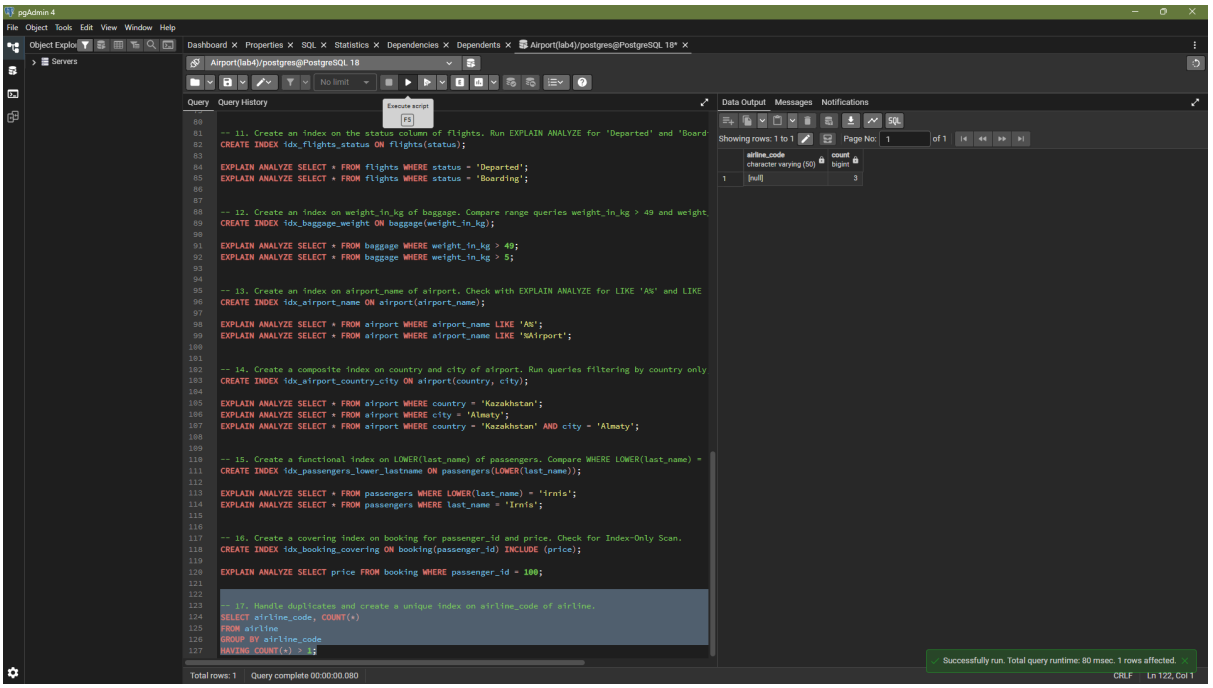
15.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Execute script
F5
Data Output Messages Notifications
Showing rows: 1 to 6 Page No: 1 of 1
QUERY PLAN
text
1 Seq Scan on passengers (cost=0.00..5.53 rows=1 width=64) (actual time=0.005..0.020 rows=1.00 loop...
2 Filter: ((last_name)::text = 'Irnis'::text)
3 Rows Removed by Filter: 201
4 Buffers: shared hit=3
5 Planning Time: 0.032 ms
6 Execution Time: 0.023 ms
-- 10. Create a composite index on booking_flight (booking_id, flight_id). Join booking, booking_flig
CREATE INDEX idx_booking_flight_composite
ON booking_flight(booking_id, flight_id);
EXPLAIN ANALYZE
SELECT f.flight_no, b.price
FROM booking b
JOIN booking_flight bf ON b.booking_id = bf.booking_id
JOIN flights f ON bf.flight_id = f.flight_id
WHERE b.price > 9000;
-- 11. Create an index on the status column of flights. Run EXPLAIN ANALYZE for 'Departed' and 'Board
CREATE INDEX idx_flights_status ON flights(status);
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Departed';
EXPLAIN ANALYZE SELECT * FROM flights WHERE status = 'Boarding';
-- 12. Create an index on weight_in_kg of baggage. Compare range queries weight_in_kg > 49 and weight_
CREATE INDEX idx_baggage_weight ON baggage(weight_in_kg);
EXPLAIN ANALYZE SELECT * FROM baggage WHERE weight_in_kg > 49;
EXPLAIN ANALYZE SELECT * FROM baggage WHERE weight_in_kg > 5;
-- 13. Create an index on airport_name of airport. Check with EXPLAIN ANALYZE for LIKE 'A%' and LIKE
CREATE INDEX idx_airport_name ON airport(airport_name);
EXPLAIN ANALYZE SELECT * FROM airport WHERE airport_name LIKE 'A%';
EXPLAIN ANALYZE SELECT * FROM airport WHERE airport_name LIKE '%Airport';
-- 14. Create a composite index on country and city of airport. Run queries filtering by country only
CREATE INDEX idx_airport_country_city ON airport(country, city);
EXPLAIN ANALYZE SELECT * FROM airport WHERE country = 'Kazakhstan';
EXPLAIN ANALYZE SELECT * FROM airport WHERE city = 'Almaty';
EXPLAIN ANALYZE SELECT * FROM airport WHERE country = 'Kazakhstan' AND city = 'Almaty';
-- 15. Create a functional index on LOWER(last_name) of passengers. Compare WHERE LOWER(last_name) =
CREATE INDEX idx_passengers_lower_lastname ON passengers(LOWER(last_name));
EXPLAIN ANALYZE SELECT * FROM passengers WHERE LOWER(last_name) = 'irnis';
EXPLAIN ANALYZE SELECT * FROM passengers WHERE last_name = 'Irnis';
Successfully run. Total query runtime: 67 msec. 6 rows affected.
Total rows: 6 Query complete 00:00:00.067
CRLF Ln 110, Col 1

16.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Execute script
Data Output Messages Notifications
Showing rows: 1 to 12 Page No: 1 of 1
QUERY PLAN
text
1 Bitmap Heap Scan on booking (cost=4.17..9.24 rows=3 width=6) (actual time=0.025..0.027 rows=3.00 loops=1)
2 Recheck Cond: (passenger_id = 100)
3 Heap Blocks: exact=2
4 Buffers: shared hit=3
5 -> Bitmap Index Scan on idx_booking_passenger_id (cost=0.00..4.17 rows=3 width=0) (actual time=0.009..0.009 rows=3.00 loo...
6 Index Cond: (passenger_id = 100)
7 Index Searches: 1
8 Buffers: shared hit=1
9 Planning:
10 Buffers: shared hit=5 read=1
11 Planning Time: 1.640 ms
12 Execution Time: 0.042 ms
Successfully run. Total query runtime: 64 msec. 12 rows affected.
Total rows: 12 Query complete 00:00:00.064
CRLF Ln 117, Col 1

17.


pgAdmin 4
File Object Tools Edit View Window Help
Dashboard Properties SQL Statistics Dependencies Dependents Airport(lab4)/postgres@PostgreSQL 18*
Airport(lab4)/postgres@PostgreSQL 18
Query Query History
Execute script
Data Output Messages Notifications
Showing rows: 1 to 1 Page No: 1 of 1
airline_code
character varying (50)
count
bigint
1 [null] 3
Successfully run. Total query runtime: 80 msec. 1 rows affected.
Total rows: 1 Query complete 00:00:00.080
CRLF Ln 122, Col 1